Key Design Decisions

A. How creature behaviour is structured (wandering and attacking)

Design 1: Implement wandering and attacking logic directly in each creature's playTurn() method.

Pros	Cons
Simple and self-contained; all logic is in one place per creature.	Duplicates wandering logic between Undead and Slime, violating DRY.
No additional classes needed.	Adding a new behaviour to a creature (e.g., fleeing) requires modifying the creature's playTurn(), violating Open-Closed Principle.
	Difficult to unit test individual behaviours in isolation.

Design 2: Use the engine's existing Behaviour<T, R> interface (Strategy pattern) with operate(Actor, Location). Concrete behaviours (WanderBehaviour, AttackBehaviour, ConsumeItemBehaviour) implement Behaviour<Actor, Action> as separate classes. Each creature holds a prioritised list of Behaviour objects and iterates through them in playTurn().

Pros	Cons
Reuses the engine's existing Behaviour<T, R> interface rather than defining a custom one, avoiding reinventing the wheel (DRY).	Introduces multiple new behaviour classes, adding structural complexity.
Each behaviour is a self-contained, reusable class (Single Responsibility Principle). WanderBehaviour is shared by both Undead and Slime.	Priority is implicit (list order), which may be less obvious than explicit if-else branching.
New behaviours can be added to any creature by simply adding to its behaviour list, without modifying existing code (Open-Closed Principle).	
Follows the Strategy pattern, making creature AI composable and testable in isolation. Consistent with the demo implementations (e.g., forest's WanderBehaviour, mars's FollowBehaviour).	
Creatures depend on the Behaviour abstraction rather than on concrete behaviour implementations (Dependency Inversion Principle).	

B. How the Undead identifies its attack targets

Design 1: Use instanceof ContractedWorker in the Undead's attack logic to identify workers.

Pros	Cons
Direct and easy to understand.	Creates a concrete dependency between AttackBehaviour and ContractedWorker, violating Dependency Inversion Principle.
	If a new actor type should also be attackable (e.g., a robot worker), AttackBehaviour must be modified, violating Open-Closed Principle.

Design 2: Use the engine's ability system. ContractedWorker enables Ability.ATTACKABLE_BY_CREATURES on itself. AttackBehaviour checks for this ability on adjacent actors via actor.hasAbility().

Pros	Cons
AttackBehaviour depends only on the Ability enum abstraction, not on any concrete actor class (Dependency Inversion Principle).	Requires adding a new enum value and granting the ability in the worker's constructor.
Any future actor that should be attackable only needs to enable ATTACKABLE_BY_CREATURES — no changes to AttackBehaviour or Undead (Open-Closed Principle).	The indirection may be less immediately obvious than a direct instanceof check.
Consistent with the existing pattern used for door unlocking (CAN_UNLOCK_DOORS) and sterilisation (HAS_STERILISATION).	

C. How the Slime consumes items from the ground

Design 1: Implement custom ground-eating logic directly inside the Slime's playTurn(), manually scanning the location for items and applying effects.

Pros	Cons
All consumption logic is in one place.	Duplicates the consumption effect logic already defined in each item's consumedBy() method, violating DRY.
	If a new consumable item is added, the Slime's logic may need updating to handle its effects, violating Open-Closed Principle.

Design 2: Create a ConsumeItemBehaviour that scans the ground for items implementing Consumable (using Location.getItemsAs(Consumable.class)), removes the item from the ground, and returns the same ConsumeAction used by workers. This fully reuses the existing Consumable interface and ConsumeAction from REQ2.

Pros	Cons
Fully reuses the Consumable interface and ConsumeAction — no separate action class needed. The same effects that apply to workers automatically apply to slimes (DRY).	The consumedBy() method in some items calls actor.getInventory().remove(this), which is a harmless no-op when the item was on the ground, but this implicit behaviour requires understanding.
New consumable items work with the Slime automatically without any modification (Open-Closed Principle).	ConsumeItemBehaviour must handle removing the item from the ground location before delegating, since the item is not in any inventory.
Follows the same Behaviour pattern as wandering and attacking, maintaining consistency across all creature actions.	

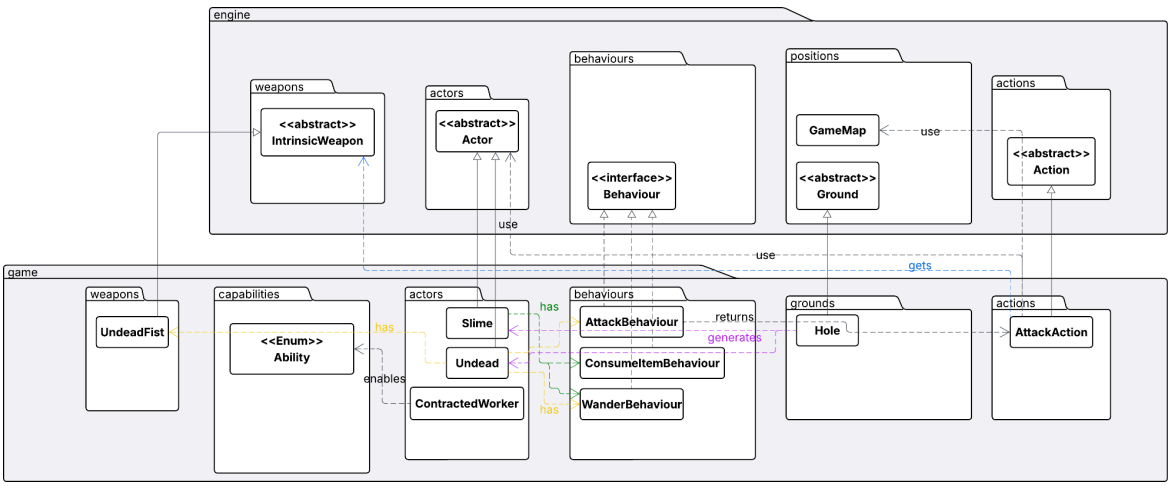
D. How the Hole spawning mechanism works

Design 1: Use a global timer in the game loop that checks all hole locations and spawns creatures.

Pros	Cons
Centralised spawning logic, easy to see all spawn events.	Requires a central manager that knows about all holes, creating tight coupling and violating Single Responsibility.
	Adding or removing holes requires modifying the central timer, violating Open-Closed Principle.

Design 2: Hole is a Ground subclass that overrides tick(Location) to maintain its own spawn counter. Every 20 turns, it creates either an Undead or Slime (equal probability) and places it on a random adjacent empty tile.

Pros	Cons
Each hole is self-contained — it manages its own counter and spawning logic (Single Responsibility Principle).	Hole depends on the concrete Undead and Slime classes for instantiation.
Adding more holes to the map requires no code changes; simply place o characters in the map string (Open-Closed for map layout).	If new creature types are added, Hole must be updated to include them in the random selection.
Leverages the engine's Ground.tick(Location) mechanism, which is called automatically each turn (DRY).	



Summary

The final design of requirement 3 utilises Design 2 of parts A, B, C, and D stated above to implement creature behaviours, target identification, slime consumption, and the hole spawning mechanism.
All creature behaviours implement the engine's existing Behaviour<Actor, Action> interface rather than a custom interface, reusing the same abstraction provided by the engine and demonstrated in its forest and mars demos. This follows the Strategy pattern, allowing creature AI to be composed from reusable, interchangeable components. Both Undead and Slime share WanderBehaviour, while Undead additionally uses AttackBehaviour and Slime uses ConsumeItemBehaviour. New behaviours can be created and assigned to any creature without modifying existing classes, satisfying the Open-Closed Principle. Each creature's playTurn() simply iterates its behaviour list and calls operate() on each, executing the first applicable action.
Target identification for the Undead uses Ability.ATTACKABLE_BY_CREATURES, continuing the ability-based decoupling pattern established in REQ1 (door unlocking) and REQ2 (sterilisation). AttackBehaviour depends on the abstract Ability enum rather than on ContractedWorker directly, satisfying the Dependency Inversion Principle.
The AttackAction delegates to the attacker's IntrinsicWeapon, which handles hit chance and damage calculation through its attack() method. The UndeadFist subclass configures 1 damage and a 10% hit rate, encapsulating weapon-specific parameters.
The Slime's ground consumption fully reuses the Consumable interface and ConsumeAction from REQ2. ConsumeItemBehaviour scans the ground for consumable items, removes the found item from the location, and returns a regular ConsumeAction. This ensures that the same effects defined in each item's consumedBy() method — including poison from rotten apples and HP reduction from toxic cookies — apply identically to slimes, without needing a separate action class for ground consumption.
The Hole ground type leverages the engine's tick(Location) lifecycle to maintain a self-contained spawn counter. Each hole independently spawns creatures every 20 turns without any centralised management, keeping the spawning logic encapsulated within the ground tile itself.